COMP3308/3608 Week 12 - Unsupervised Learning

Author
K. Yu
2026

Contents

1 Clustering Overview

A **clustering** is a function $f : \{1, \ldots, n\} \to \{1, \ldots, k\}$ assigning each example to one of k k groups. Writing $K_j = \{i : f(i) = j\}$, the resulting partition $\mathcal{K} = \{K_1, \ldots, K_k\}$ satisfies

$$\bigcup_{j=1}^{k} K_j = \{1, \ldots, n\}, \quad K_i \cap K_j = \emptyset \text{ for } i \neq j.$$

This is **hard clustering**. **Soft clustering** instead returns a responsibility distribution $\gamma_{ij} \in [0, 1]$ with $\sum_j \gamma_{ij} = 1$.

The qualitative goal is to maximise *cohesion* (intra-cluster similarity) and *separation* (inter-cluster dissimilarity). Both are operationalised through a distance $d : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_{\geq 0}$.

1.1 Supervised vs. Unsupervised Learning

- Supervised learning observes $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$ and seeks $\hat{f}$ minimising a *known* loss $L(\hat{f}(\mathbf{x}), y)$.
- Unsupervised learning observes only $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^{n}$ and seeks structure (clusters, density, low-dimensional manifold). There is no ground-truth loss; we substitute a *surrogate* objective (SSE, log-likelihood, modularity, etc.).

1.2 Geometric preliminaries

Distances

- **Euclidean (L_2)**: $d_2(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{d} (x_j - z_j)^2}$
- **Manhattan (L_2)**: $d_1(\mathbf{x}, \mathbf{z}) = \sum_{j=1}^{d} |x_j - z_j|$
- **Cosine dissimilarity**: $d_{\cos}(\mathbf{x}, \mathbf{z}) = 1 - \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\|_2 \|\mathbf{z}\|_2}$
- **Mahalanobis**: $d_M(\mathbf{x}, \mathbf{z}) = \sqrt{(\mathbf{x} - \mathbf{z})^\top \Sigma^{-1} (\mathbf{x} - \mathbf{z})}$ accounts for feature correlation.

Practical caveat. Distances are scale-sensitive; features should be standardised or min-max normalised before clustering.

Centroid and Medoid

For a cluster K with $|K| = N$:

- **Centroid**: $\mathbf{c} = \frac{1}{N} \sum_{i \in K} \mathbf{x}^{(i)}$
- **Medoid**: $\mathbf{m} = \text{argmin}_{\mathbf{x}^{(i)} \in K} \sum_{j \in K} d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$

The centroid is the L_2 Fréchet mean and need not be a data point; the medoid is restricted to the dataset and is robust to outliers.

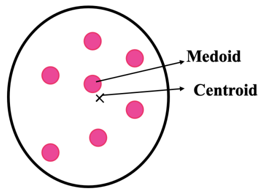

Figure 1: Centroid and medoid.

Distance between clusters

For clusters K_i, K_j:

Linkage	**Definition**
Single (MIN)	$\min_{\mathbf{x}\in K_i,\mathbf{z}\in K_j} d(\mathbf{x},\mathbf{z})$
Complete (MAX)	$\max_{\mathbf{x}\in K_i,\mathbf{z}\in K_j} d(\mathbf{x},\mathbf{z})$
Average	$\frac{1}{d}\sum_{j=1}^{d}(x_j - z_j)$
Centroid	$d(\mathbf{c}_i, \mathbf{c}_j)$
Ward	$\frac{\lvert K_i\rvert\lvert K_j\rvert}{\lvert K_i\rvert+\lvert K_j\rvert}[d_2(\mathbf{x},\mathbf{z})]^2$

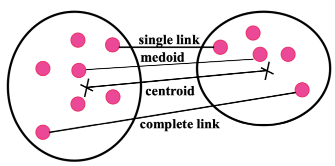

Figure 2: Distance metrics between clusters.

2 Evaluating a Clustering

The fundamental difficulty: without labels there is no unique ”right answer”, so we use *internal* indices.

2.1 Davies-Bouldin Index

Let

$$\bar{d} = \frac{1}{|K_i|}\sum_{\mathbf{x}\in K_i}\|\mathbf{x} - c_i\|_2^2$$

denote the mean-squared spread of cluster i. Define

$$\mathrm{DB} = \frac{1}{k}\sum_{i=1}^{k} \max\left\{\frac{\bar{d}(\mathbf{x}, \mathbf{c}_i) + \bar{d}(\mathbf{x}, \mathbf{c}_j)}{d(\mathbf{c}_i, \mathbf{c}_j)} : j \neq i\right\}$$

Numerator measures *spread*; denominator measures *separation.* The "max" picks the *worst* neighbour for each cluster (its closest, most-overlapping competitor). **Small DB $\implies$ good clustering**.

2.2 Silhouette Coefficient

For point $\mathbf{x}^{(i)} \in K_j$:

$$a(i) = \frac{1}{|K_j| - 1}\sum_{l \in K_j \setminus \{i\}} d(\mathbf{x}^{(i)}, \mathbf{x}^{(l)}), \quad b(i) = \min\left\{\frac{1}{|K_m|}\sum_{l \in K_m} d(\mathbf{x}^{(i)}, \mathbf{x}^{(l)}) : m \neq j\right\}$$

The silhouette is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1].$$

Interpretation: $s(i) \approx 1$ means $\mathbf{x}^{(i)}$ is well inside its cluster; $s(i) \approx 0$ means it sits on the boundary $s(i) < 0$ suggests misclassification. The average silhouette $\bar{s} = \frac{1}{n}\sum_i s(i)$ is reported for the whole clustering and is often used to *choose* k.

3 Taxonomy of Algorithms

- **Partitional** — produce a single flat partition. Require k (k-means, k-medoids) or a threshold (nearest-neighbour, SOM).
- **Hierarchical** — produce a dendrogram (nested partitions). Agglomerative (bottom-up) or divisive (top-down). k chosen *post hoc* by cutting the dendrogram.
- **Density-based** — clusters are maximal connected regions of high density (DBSCAN, OPTICS). k is emergent.
- **Model-based** — assume a generative model, fit by MLE/EM (Gaussian mixtures).
- **Fuzzy** — soft assignments (fuzzy c-means).

4 k-Means

Algorithm 1 Lloyd's iteration

Require: $\{\mathbf{x}^{(i)}\}_{i=1}^{n}$, integer k.

  Initialise $\mathbf{c}_1, \ldots, \mathbf{c}_k \in \mathbb{R}^d$ (e.g. random data points)

  loop Repeat until convergence (centroids stop changing)

    for $i = 1, \ldots, n$ **do**

      $z^{(i)} \leftarrow \operatorname{argmin}_{j \in \{1,\ldots,k\}} \|\mathbf{x}^{(i)} - \mathbf{c}_j\|_2^2$

      $K_j \leftarrow \{i : z^{(i)} = j\}$

      $\mathbf{c}_j \leftarrow \frac{1}{|K_j|} \sum_{i \in K_j} \mathbf{x}^{(i)}$

    end for

  end loop

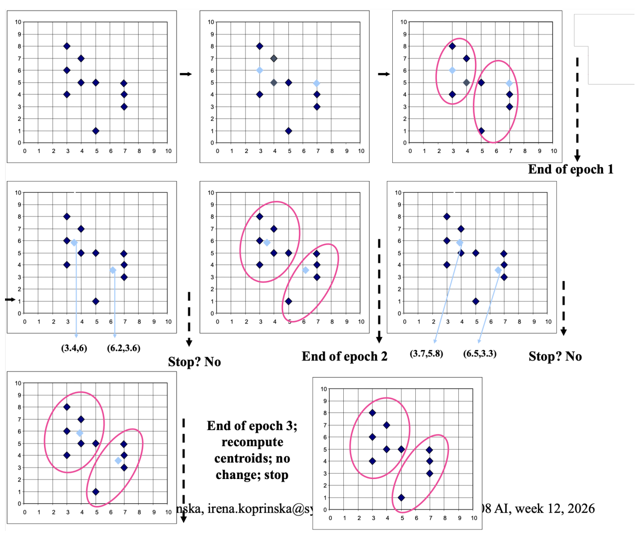

Figure 3: Workflow of k-means.

4.1 Optimisation view

k-means is exactly *coordinate descent* on the **sum-of-squared errors** (SSE) objective

$$J(\{\mathbf{c}_j\}, \{z^{(i)}\}) = \sum_{j=1}^{k} \sum_{i \in K_j} \|\mathbf{x}^{(i)} - \mathbf{c}_j\|_2^2.$$

- **Assignment step** fixes $\{\mathbf{c}_j\}$ and minimises J over the discrete partition. For each i, choose the closest centroid, this is the global minimum with respect to $z^{(i)}$.

- **Update step** fixes $\{z^{(i)}\}$ and minimises over $\{\mathbf{c}_j\}$. Setting

$$\frac{\partial J}{\partial \mathbf{c}_j} = -2 \sum_{i \in K_j} (\mathbf{x}^{(i)} - \mathbf{c}_j) = \mathbf{0}$$

gives

$$\mathbf{c}_j = \frac{1}{|K_j|} \sum_{i \in K_j} \mathbf{x}^{(i)}.$$

Thus, the centroid is the L_2-optimal representative.

Because each step weakly decreases J and $J \geq 0$, the algorithm converges in a *finite* number of iterations (the partition space is finite). However the limit is a **local** minimum; the problem is NP-hard in general.

Complexity

- Time: $O(t \cdot k \cdot n \cdot d)$ — t iterations, each computes kn distances of dimension d.
- Space: $O\big((n+k)d\big)$.

4.2 Sensitivity to initialisation

Different seeds can produce very different local minima. Mitigations:

- Multiple random restarts, keep the run with lowest J.
- **k-means** initialisation: pick $\mathbf{c}_1$ uniformly at random; pick subsequent $\mathbf{c}_j$ with probability proportional to $\min\{\|\mathbf{x}^{(i)} - \mathbf{c}_l\|_2^2 : l < j\}$. This gives an $O(\log k)$ approximation guarantee in expectation.

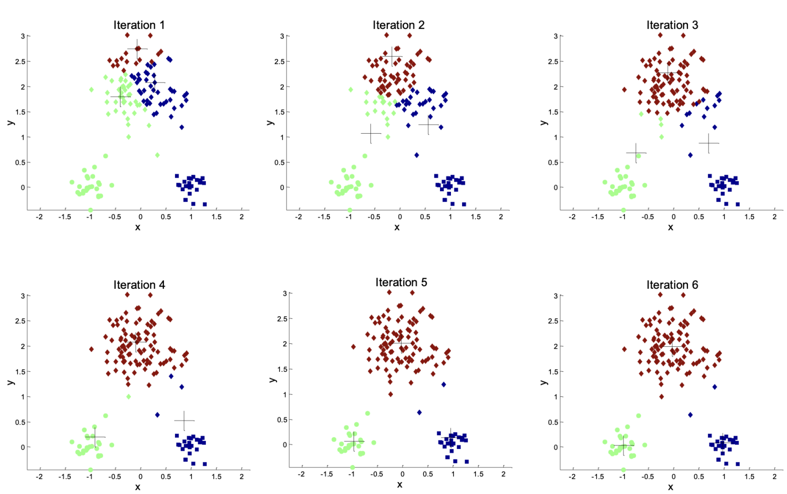

Figure 4: Good initial centroids.

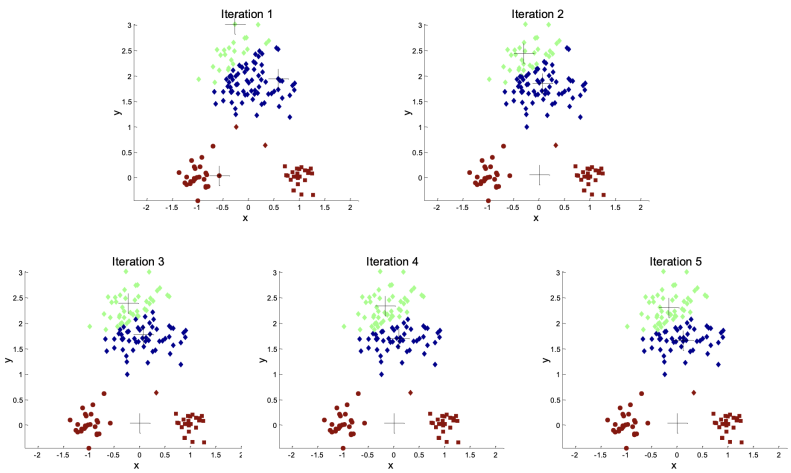

Figure 5: Poor initial centroids.

4.3 Limitations

- Assumes **spherical, isotropic, comparably-sized** clusters (it draws Voroni cells, which are convex polytopes).
- Sensitive to outliers (a single far-away point can drag a centroid arbitrarily far).
- Requires k a priori — use silhouette / DB / elbow on $J(k)$ to choose.
- Categorical data must first be encoded numerically (k-modes is an alternative for raw categorical).

4.4 Worked example

With distance matrix on items $\{A, B, C, D, E\}$ (shown below) and initial centroids $\{B, C\}$:

Item	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

- Assignment 1: A closer to B ($1 < 2$); D, E closer to C; but E is equidistant to B, C. Tie-breaking gives $K_1 = \{A, B\}$, $K_2 = \{C, D, E\}$ (or symmetric).
- Update centroids and keep iterate.

4.5 Application: Vector Quantisation

Image compression: split a 1024×1024 greyscale image into 2×2 blocks. Each block is a vector $\mathbf{x} \in \mathbb{R}^4$ (so $n = 512^2 = 262,144$). Run k-means with k centroids $\{\mathbf{c}_1, \dots, \mathbf{c}_k\}$ (the **codebook**); replace each block by its centroid index.

Storage cost (bits):

$$\underbrace{k \cdot 4 \cdot 8}_{\text{codebook}} + \underbrace{n \cdot \lceil \log_2 k \rceil}_{\text{indices}}.$$

Compression ratio vs original $1024^2 \cdot 8$ bits:

- $k = 200$, ≈ 0.239
- $k = 4$, ≈ 0.063 (but visible degradation)

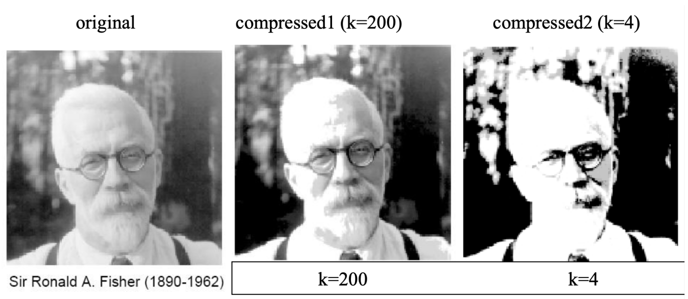

Figure 6: Fisher portrait comparison — original / $k = 200$ / $k = 4$.

5 k-Medoids (COMP3608 only)

5.1 Objective

k-Mediods is also called **Partitioning Around Medoids** (PAM). Replace the Euclidean-squared cost by the **sum of distances to the medoid**:

$$J_{\text{PAM}} = \sum_{j=1}^{k} \sum_{i \in K_j} d(\mathbf{x}^{(i)}, \mathbf{m}_j).$$

The key differences from k-means:

1. The representative $\mathbf{m}_j$ is constrained to lie in the dataset.
2. The distance is L_1-style, which is more **robust** to outliers.

5.2 Algorithm (hill-climbing swap heuristic)

In high-level:

1. Pick k initial methods.
2. Assign every non-medoid to its closest medoid.
3. For each medoid $\mathbf{m}_j$ and each non-medoid $\mathbf{x}^{(i)}$: tentatively swap, recompute J_{PAM}. There are $k(n-k)$ candidate swaps per iteration.
4. Accept the best swap if it decreases cost; else stop.

This is **hill-climbing search** on a state space whose states are sets of k data points, with the partial cost as heuristic.

Complexity

- Per iteration: $O(k(n-k)n)$ for evaluating all candidate swaps.
- Space: $O(n^2)$ if the pairwise distance matrix is precomputed.
- Significantly more expensive than k-means, so PAM is impractical for large n (faster variants: CLARA, CLARANS).

5.3 When k-medoids beats k-means

- The data is only given as a **dissimilarity matrix**, with no underlying vector representation. k-means requires coordinates to compute means; k-medoids does not.
- Robust statistics — heavy-tailed noise, outliers.

6 Nearest-Neighbour (Single-Pass) Clustering

A streaming-friendly algorithm with one hyper-parameter t:

Algorithm 2 Nearest-neighbour clustering

```
Require: {x^(i)}_{i=1}^n, threshold t
  K_1 ← {x^(1)}
  k ← 1
  for i = 2, ..., n do
      (m, j) ← argmin_{m,j} d(x^(i), x^(j)), where x^(j) ∈ K_m
      if d(x^(i), x^(j)) ≤ t then
          K_m ← K_m ∪ {x^(i)}
      else
          k ← k + 1
          K_k ← {x^(i)}
      end if
  end for
```

- Distance used: **single-link** (MIN over the cluster).
- **Order-dependent**: different orderings of $\{\mathbf{x}^{(i)}\}$ can yield different clusterings.
- **One-pass**: makes it suitable for streaming or "data that changes over time."
- Time/space: $O(n^2)$ in the worst case (each new point may compare to every prior point).
- The threshold t implicitly controls the number of clusters; small $t \implies$ many singletons, large $t \implies$ one big cluster.

7 Hierarchical Clustering

Produces a **dendrogram** — a binary tree of nested partitions. Cutting the tree at height h yields a clustering; thus a single run delivers a *family* of clusterings parametrised by h.

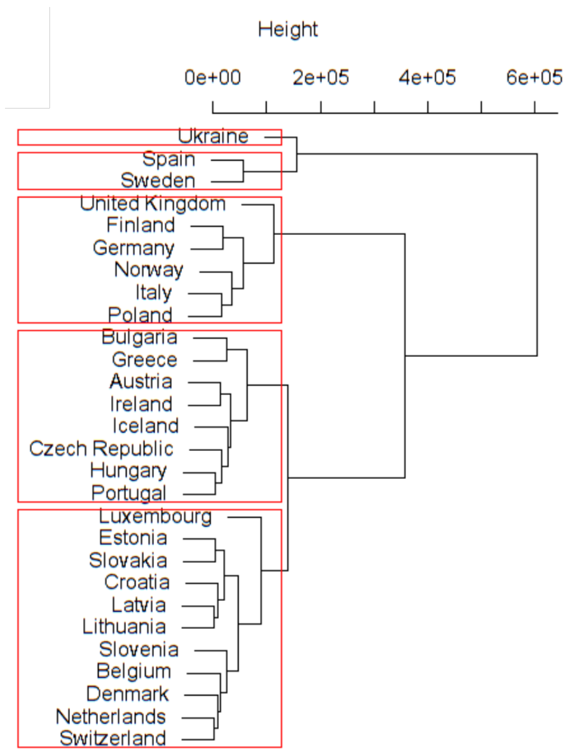

Figure 7: European-countries dendrogram.

7.1 Agglomerative (Bottom-Up)

1. Start with n singleton clusters. Compute the $n \times n$ distance matrix D with $[D]_{ij} = d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.
2. Repeat until one cluster remains:

- Find $(K_a, K_b) = \text{argmin}_{i \neq j} D(K_i, D_j)$ under the chosen linkage.
- Merge: $K_{a\cup b} \leftarrow K_a \cup K_b$; remove rows/columns for K_a, K_b from D; add a row/column for $K_{a\cup b}$.
- Record the merge height in the dendrogram.

7.2 Lance-Williams Update Formula

After merging K_a, K_b into $K_{a\cup b}$, the distance to any other cluster K_c can be expressed as

$$d(K_{a\cup b}, K_c) = \alpha_a d(K_a, K_c) + \alpha_b d(K_b, K_c) + \beta d(K_a, K_b) + \gamma |d(K_a, K_c) - d(K_b, K_c)|,$$

with linkage-specific coefficients $(\alpha_a, \alpha_b, \beta, \gamma)$:

Linkage	α_a	α_b	β	γ
Single	$1/2$	$1/2$	0	$-1/2$
Complete	$1/2$	$1/2$	0	$1/2$
Average	$\frac{\lvert K_a\rvert}{\lvert K_a\rvert+\lvert K_b\rvert}$	$\frac{\lvert K_b\rvert}{\lvert K_a\rvert+\lvert K_b\rvert}$	0	0

This unification means a single implementation handles all linkages.

7.3 Single vs. Complete Link

- **Single link** tends to produce *elongated, chain-like* clusters (the "chain effect": two clusters merge whenever *any* pair of their points is close). Sensitive to noise.
- **Complete link** produces *tighter, more compact, spherical-like* clusters. Less affected by noise but biased against elongated true structure.

7.4 Divisive (Top-Down)

Start with all points in one cluster; recursively split. A natural splitting rule (DIANA) is to grow a "splinter group" by repeatedly moving the most-dissatisfied point. Divisive methods are less common because the split decision at the root must consider $2^{n-1} - 1$ possible bipartitions.

7.5 Complexity

- Space: $O(n^2)$ for the distance matrix D.
- Time: naively $O(n^3)$; with sorted distances/heap, single-link can be done in $O(n^2)$ and others in $O(n^2 \log n)$.
- **Not incremental** — adding a new point in principle requires rerunning.

7.6 Worked single-link example

With the same $\{A, B, C, D, E\}$ distance matrix shown in 4.4:

- Level 1 ($d = 1$): merge $\{A, B\}$ and $\{C, D\}$, both pairs at distance 1.
- Level 2 ($d = 2$): single-link distance between $\{A, B\}$ and $\{C, D\}$ is

$$\min\{d(A, C), d(A, D), d(B, C), d(B, D)\} = 2,$$

  so merge.
- Level 3($d = 3$): merge $\{A, B, C, D\}$ with $\{E\}$.

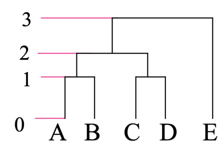

Figure 8: The resulting dendrogram in the worked single-link example.

8 DBSCAN (Not Examinable)

Density-Based Spatial Clustering of Applications with Noise (DBSCAN). Clusters are maximal connected regions where points are densely packed; sparse points are labelled *noise*. Two hyperparameters: a radius $\varepsilon > 0$ and an integer MinPts.

8.1 Definitions

- **ε-neighbourhood:** $N_\varepsilon(\mathbf{x}) = \{\mathbf{x}' \in \mathcal{D} : d(\mathbf{x}, \mathbf{x}') \leq \varepsilon\}$.
- **Core point:** $|N_\varepsilon(\mathbf{x})| \geq \text{MinPts}$.
- **Directly density-reachable:** $\mathbf{x}'$ is d.d.r. from core $\mathbf{x}$ if $\mathbf{x}' \in N_\epsilon(\mathbf{x})$.
- **Density-reachable:** the transitive closure of d.d.r. via a chain of core points.
- **Density-connected:** $\mathbf{x}, \mathbf{x}'$ are d.c. if there exists a core $\mathbf{q}$ from which both are density-reachable.

A **cluster** is a maximal density-connected set. Non-core, non-reachable points are **noise**.

8.2 Algorithm sketch

For each unvisited $\mathbf{x}^{(i)}$, if $|N_\varepsilon(\mathbf{x}^{(i)})| \geq \text{MinPts}$, start a new cluster and BFS-expand through density-reachable points; otherwise mark as noise (may be reabsorbed later as a border point).

8.3 Properties

- Discovers clusters of **arbitrary shape** (not just convex).
- No need to specify k.
- robust to outliers (they become noise).
- Sensitive to $(\varepsilon, \text{MinPts})$ and struggles when clusters have very different densities (variant: OPTICS).
- Complexity $O(n \log n)$ with spatial indexing (e.g. R-tree), $O(n^2)$ otherwise.

9 Probabilistic Clustering: Gaussian Mixture Models (Not Examinable)

9.1 Model

Assume each $\mathbf{x}^{(i)}$ is drawn from one of k Gaussian components with mixing weights π_j:

$$p(\mathbf{x} \mid \theta) = \sum_{j=1}^{k} \pi_j \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_j, \Sigma_j), \quad \sum_j \pi_j = 1,\ \pi_j \geq 0.$$

Parameters $\theta = \{\pi_j, \boldsymbol{\mu}_j, \Sigma_j\}_{j=1}^{k}$ are fit by maximum likelihood:

$$\theta^* = \operatorname*{argmax}_{\theta} \sum_{i=1}^{n} \log \left[\sum_{j=1}^{k} \pi_j \mathcal{N}(\mathbf{x}^{(i)} \mid \boldsymbol{\mu}_j, \Sigma_j) \right].$$

The log-sum makes direct optimisation intractable; we use **EM**.

9.2 EM Algorithm

Introduce latent variables $z^{(i)} \in \{1, \ldots, k\}$ indicating component membership.

E-step. Compute responsibilities

$$\gamma_{ij} = p(z^{(i)} = j \mid \mathbf{x}^{(i)}, \theta) = \frac{\pi_j \mathcal{N}(\mathbf{x}^{(i)} \mid \boldsymbol{\mu}_j, \Sigma_j)}{\sum_{l=1}^{k} \pi_l \mathcal{N}(\mathbf{x}^{(i)} \mid \boldsymbol{\mu}_l, \Sigma_l)}.$$

M-step. Update parameters using soft counts $N_j = \sum_i \gamma_{ij}$:

$$\boldsymbol{\mu}_j = \frac{1}{N_j} \sum_{i=1}^{n} \gamma_{ij} \mathbf{x}^{(i)}, \quad \Sigma_j = \frac{1}{N_j} \sum_{i=1}^{n} \gamma_{ij} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_j)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_j)^\top, \quad \pi_j = \frac{N_j}{n}.$$

EM monotonically increases the log-likelihood and converges to a local maximum.

9.3 Connection to k-means

Take $\Sigma_j = \sigma^2 I$ for all j, with $\sigma \to 0^+$. Then

$$\gamma_{ij} = \frac{\pi_j \exp\Big(-\frac{\|\mathbf{x}^{(i)}-\boldsymbol{\mu}_j\|^2}{2\sigma^2}\Big)}{\sum_l \pi_l \exp\Big(-\frac{\|\mathbf{x}^{(i)}-\boldsymbol{\mu}_l\|^2}{2\sigma^2}\Big)} \xrightarrow{\sigma\to 0} \mathbf{1}\Big\{j = \operatorname*{argmin}_l \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_l\|^2\Big\}$$

Responsibilities become hard assignments; the M-step reduces to averaging, which is exactly k-means. So **k-means is the zero-variance, hard-assignment, isotropic-covariance limit of GMM**.

9.4 Strengths

- Soft assignments / probabilistic uncertainty.
- Elliptical clusters via Σ_j.
- A principled framework (likelihood, BIC for choosing k).

10 Principal Component Analysis (Not Examinable)

PCA is a dimensionality-reduction preprocessing step that radically improves clustering when d is large.

10.1 Setup

Assume the data is centred, i.e. $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}^{(i)} = \mathbf{0}$ (subtract the mean if not). Define the sample covariance matrix

$$S = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}^{(i)}(\mathbf{x}^{(i)})^\top = \frac{1}{n}X^\top X \in \mathbb{R}^{d\times d},$$

where $X \in \mathbb{R}^{n\times d}$ stacks the data rows.

10.2 The First PC: Maximum-Variance Projection

Find a unit direction $\mathbf{w} \in \mathbb{R}^d$ maximising the variance of the projection $\mathbf{w}^\top\mathbf{x}$:

$$\max_{\mathbf{w}\in\mathbb{R}^d} \mathbf{w}^\top S\mathbf{w} \quad \text{s.t.} \quad \|\mathbf{w}\| = 1.$$

Lagrangian $\mathcal{L} = \mathbf{w}^\top S\mathbf{w} - \lambda(\mathbf{w}^\top\mathbf{w} - 1)$, stationary point $S\mathbf{w} = \lambda\mathbf{w}$. So $\mathbf{w}$ is an **eigenvector** of S, and the optimum is attained at the eigenvector with the largest eigenvalue λ_1.

10.3 Subsequent PCs and SVD

Iteratively maximise variance subject to orthogonality to previous PCs — this yields the eigenvectors $\mathbf{w}_1, \dots, \mathbf{w}_d$ of S ordered by $\lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_d \ge 0$.

Equivalently, the **SVD** $X = U\Sigma V^\top$ gives the PCs as the columns of V and the singular values $\sigma_i = \sqrt{n\lambda_i}$.

10.4 Reconstruction-error view

Projecting onto the top r PCs minimises the expected squared reconstruction error:

$$W_r^* = \underset{W \in \mathbb{R}^{d \times r}, W^\top W = I_r}{\operatorname{argmin}} \sum_{i=1}^{n} \|\mathbf{x}^{(i)} - WW^\top \mathbf{x}^{(i)}\|^2,$$

with optimum $W_r^* = \begin{bmatrix} \mathbf{w}_1 & \cdots & \mathbf{w}_r \end{bmatrix}$ (Eckart-Young).

10.5 PCA before clustering

- Decorrelates features; isotropies the data so Euclidean distance is more meaningful.
- Reduces the "curse of dimensionality": in hgih d, all pairwise distances concentrate, hurting k-means.
- Caveat: PCA preserves *variance*, not necessarily *cluster structure*; for cluster discovery one may prefer LDA-style or t-SNE/UMAP embeddings.

10.6 PCA-k-means equivalence

A classical result (Ding & He, 2004): the *continuous relaxation* of k-means cluster indicators is exactly given by the top $k-1$ principal components. PCA thus provides a lower bound on the k-means SSE, and PCA-space clustering is often used as initialisation.

11 Cross-Cutting Themes

11.1 How to choose k

- **Elbow method.** Plot $J(k)$ (k-means SSE) vs k; pick the inflection where adding clusters yields diminishing returns.
- **Silhouette.** Pick k maximising mean silhouette.
- **Gap statistic.** Compare $\log J(k)$ to its expectation under a uniform reference null.
- **BIC / AIC** for GMM: penalised likelihood.

11.2 The Curse of Dimensionality

As $d \to \infty$ with n fixed, all pairwise distances concentrate:

$$\frac{\max_{i,j} \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\| - \min_{i,j} \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|}{\min_{i,j} \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|} \xrightarrow{d \to \infty} 0$$

under mild conditions. Distance-based clustering becomes meaningless. Mitigations: dimensionality reduction (PCA, autoencoder), subspace clustering, learned metrics.

11.3 Algorithm selection cheatsheet

Need	**Algorithm**
Fast, large n, roughly spherical clusters	k-means (++init)
Only have pairwise distances	k-medoids, hierarchical
Outliers, robust representative	k-medoids
Arbitrary-shape clusters, noise points	DBSCAN
Soft assignments, elliptical clusters	GMM/EM
Interpretable nested structure (taxonomies)	Agglomerative hierarchical
Streaming / online	Nearest-neighbour single-pass
High dimensional	PCA preprocessing, then any of the above